arXiv:2611.14848v1 [cs.OS] 10 Nov 2026

TIDEWAY: PHASE-AWARE TIERED MEMORY MANAGEMENT FOR MOE INFERENCE ON CONSUMER DEVICES

Yifei Wang [1] **Tsun-yu Yang** [1] **Yechen Xu** [1] **Hancheng Ye** [1] **Qinsi Wang** [1]
Baijun Zhou [1] **Tingjun Chen** [1] **Hai "Helen" Li** [1] **Danyang Zhuo** [1] **Yiran Chen** [1]

ABSTRACT

Large language model (LLM) agents are rapidly becoming a dominant paradigm for AI applications. Recently, there is a growing demand to deploy large-scale Mixture-of-Experts (MoE) models locally on consumer PCs to balance data privacy, API costs, and inference efficiency. However, unlike datacenter servers with abundant DRAM, consumer devices are tightly constrained in both GPU VRAM and host RAM, forcing massive MoE weights to spill across VRAM, host RAM, and SSD. Under such spilling, we observe that existing systems fail to fully utilize this memory hierarchy, treating the tiers as independent caches and thrashing in inference.

We present TIDEWAY, a system for efficient inference of large MoE models on memory-constrained consumer devices. Our core insight is to manage host RAM and GPU VRAM as a unified memory pool. Within this pool, TIDEWAY couples phase-aware memory layouts, which adapt to the distinct access patterns of prefill and decode, with an execution engine that optimizes CUDA Graph execution under expert paging. Our evaluations on representative consumer hardware with frontier MoE models demonstrate that TIDEWAY significantly outperforms state-of-the-art systems. On average, TIDEWAY achieves speedups of 1.97× in end-to-end latency, 2.40× in Time-to-First-Token (TTFT), and 1.61× in Time-Per-Output-Token (TPOT) against the fastest baseline.

Our source code is publicly available at https://github.com/Afterglow231/Tideway.

1 INTRODUCTION

Large language models (LLMs) increasingly serve as the control core of autonomous agents, now in everyday use for research (OpenAI, 2025b), computer use (Xie et al., 2024), and, most visibly, software engineering (Anthropic, 2025; OpenAI, 2025a). These interactive agents are mostly powered by cloud APIs. Meanwhile, open-weight mixture-of-experts (MoE) models now offer strong reasoning and coding capability while activating only a fraction of their parameters per token (Yang et al., 2025; DeepSeek-AI, 2026), and self-hosting them brings data privacy, reduces the token costs from long-running tasks, and permits domain-specific adaptation (Song et al., 2024; Erdogan et al., 2024). Consequently, users are increasingly motivated to run these agents locally and cheaply, keeping sensitive data on-device.

Recently, open-source engines such as FreeToken (Yang et al., 2026) and llama.cpp (Gerganov et al., 2023) are bringing frontier-scale MoE models to personal PCs with consumer GPUs. However, even high-end consumer GPUs lack sufficient VRAM to hold the largest models, shifting much of the storage burden to host RAM. Particularly, for many users, a capable GPU and 16–64 GiB of system RAM are sufficient for everyday computing. However, upgrading to hundreds of gigabytes of RAM solely to accommodate large AI model weights can entail a substantial additional investment. Growing AI demand further increases this cost because GPU memory and system RAM compete for the same DRAM manufacturing capacity, driving up memory prices (TrendForce, 2026). On such PCs, weights spill beyond VRAM into host RAM and, when the combined memory budget is insufficient, extend to SSD.

A central challenge is *how to minimize costly weight transfers while sustaining efficient GPU execution under expert paging*. The activated experts are decided at runtime, so cross-tier transfers land on the critical path of every forward pass. This cost appears in both phases. During prefill, a long prompt activates nearly every expert of every layer, so the whole model streams into VRAM before the first token is produced, inflating Time-to-First-Token (TTFT). During decode, each step touches only a few experts, but any missing expert stalls the decoding step and lengthens Time-Per-Output-Token (TPOT). Across long generations and successive turns, these stalls accumulate into end-to-end latency. Expert layout across tiers thus decides not only whether the model fits but also its inference performance.

[1]Duke University, Durham, North Carolina, USA.

Preprint. Under review.

Existing inference systems struggle with this challenge, as their memory assumptions and optimization strategies do not match a consumer PC. FreeToken optimizes prefill and decode separately but assumes the complete expert pool resides in host RAM; its VRAM cache stores copies of experts already held in RAM, and the design has no fallback once the pool exceeds host memory. llama.cpp offers offloading and hybrid CPU–GPU execution but delegates paging to the OS through memory-mapped files, which operate at page rather than expert granularity and thrash once the working set reaches SSD. FlashMoE (Kim et al., 2026) supports SSD-backed inference, but its learned replacement policy still thrashes under full-model prefill scans. SSD-LLaMA (Liang et al., 2026) cuts expert-loading overhead with compression, asynchronous transfers, and cross-tier caching, yet keeps one layout fixed across phases, leaving memory underutilized and incurring avoidable transfers. Ultimately, *none of these systems treats the constrained GPU and host memory as a unified pool, failing to utilize the full potential of the memory hierarchy on consumer PCs*.

We design and implement TIDEWAY, a system for efficient large MoE inference on consumer PCs, which couples a phase-aware memory manager with an execution engine. Instead of treating host RAM as a swap space behind VRAM, the manager combines GPU and host memory into a unified pool backed by the SSD. To maximize effective capacity and reduce expert misses, TIDEWAY deduplicates experts across the VRAM and RAM tiers and exploits bi-directional PCIe bandwidth, selectively returning evicted GPU experts to host RAM to avoid subsequent SSD reloads.

Within the pool, we introduce a phase-aware strategy. TIDEWAY maintains separate memory layouts for streaming prefill and sparse decode, and switches between them at the phase boundary. This adaptation preserves optimal memory states and actively prevents cross-tier thrashing during phase transitions. To sustain efficient GPU execution as expert residency changes, TIDEWAY preserves CUDA Graph replay for reusable portions of decode computation. The engine dynamically fetches out-of-core experts and retains graph-based execution wherever possible, limiting per-token host orchestration and kernel-launch overhead. Together, these mechanisms improve both prefill and decode performance without requiring the full expert pool to fit in host memory.

We evaluate TIDEWAY on an NVIDIA RTX 5090 with 32 GB of VRAM under host RAM budgets of 16–128 GiB, using three open-weight MoE models: Qwen3.6-35B-A3B (Qwen Team, 2026), Gemma4-26B-A4B (Gemma Team, 2026), and DeepSeek-V4-Flash. The evaluation covers 270 fixed-shape configurations with input lengths of 4K to 128K tokens and output lengths of 128 to 2K tokens, and trace-driven replays of eight multi-turn coding-agent sessions from TraceLab (Zhu et al., 2026). Across all fixed-shape configurations, TIDEWAY achieves 1.97× average and up to 3.18× end-to-end speedups over the fastest baseline, with 2.40× and 1.61× average gains in TTFT and TPOT. On TraceLab replays of coding-agent sessions, TIDEWAY further reduces total request processing time by up to 3.14×.

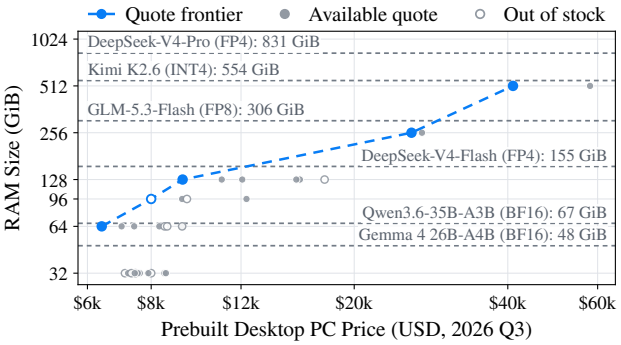

Figure 1. Prebuilt desktop quotes by price and RAM size (dots), and the weight size of open-weight MoE models (dashed lines).

Our paper makes the following contributions:

- We characterize the performance degradation of existing MoE inference systems under limited host RAM, identifying SSD traffic and execution overhead as key bottlenecks.
- We design TIDEWAY, coupling a unified, phase-aware memory manager with an execution engine that preserves CUDA Graph replay under expert paging, reducing SSD accesses and per-token overhead.
- We evaluate TIDEWAY on frontier-scale MoE models. Against the fastest baselines, TIDEWAY achieves 1.97× average end-to-end speedup on fixed-shape workloads and up to 3.14× speedup on coding-agent traces.

2 MOTIVATION

2.1 Memory on Consumer PCs

Deploying a frontier-scale MoE model locally offers compelling privacy and cost benefits, but a consumer PC rarely has the memory to hold one. Four 2026 datasets, RTX 5090 desktops on sale from system integrators, 3DMark submissions (UL Solutions, 2026) and BOINC hosts (BOINC, 2026) with an RTX 5090, and the Steam Hardware Survey (Valve Corporation, 2026), tell the same story: 16–64 GiB of RAM is the mainstream, covering 64–84% of the three RTX 5090 populations and 99% of Steam machines, while more than 128 GiB appears in at most 11%, with the largest share among BOINC hosts. Even owners of the top consumer GPU stop at 64 GiB.

Purchasing a desktop with substantially more RAM requires a different system budget. Each point in Figure 1 is a prebuilt desktop quote plotted by price and RAM size, and the blue line traces the cheapest quote at each capacity. The

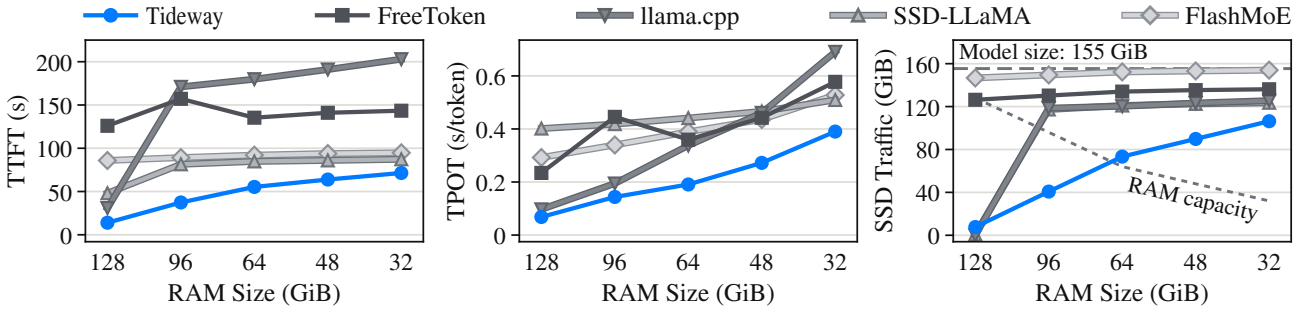

Figure 2. TTFT, TPOT, and SSD traffic of DeepSeek-V4-Flash-FP4 on an RTX 5090 as host RAM shrinks from 128 to 32 GiB, replaying realistic agent traces in TraceLab with prefill chunk size set to 8K tokens.

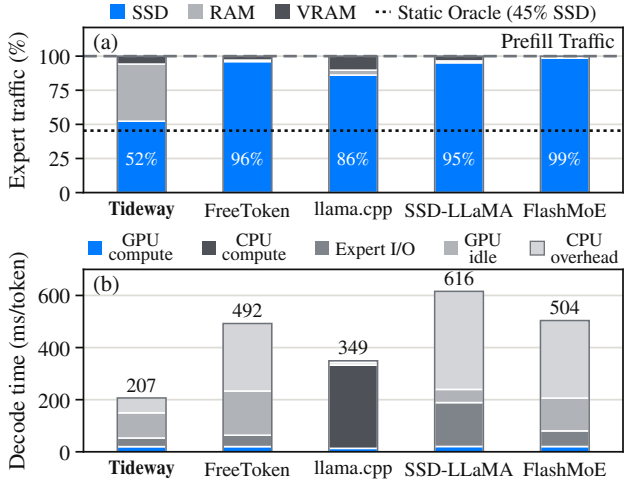

Figure 3. (a) Prefill expert traffic and (b) breakdown of per-token decode time for DeepSeek-V4-Flash on an RTX 5090 with 64 GiB of host RAM (32K-token prompt, 1K output tokens).

frontier stays under $10k up to 128 GiB and then bends sharply, to about $26k at 256 GiB and $41k at 512 GiB. The dashed lines mark open-weight MoE models, which show that even expensive configurations cannot accommodate the full range of frontier MoE models. Provisioning sufficient RAM for full model residency is therefore a costly requirement, not a realistic default for consumer devices.

2.2 MoE Inference with Insufficient RAM

When GPU and host memory cannot retain the full model, loading missing experts from SSD becomes part of the inference path. Figure 2 quantifies this cost by replaying agent traces with DeepSeek-V4-Flash-FP4 on an RTX 5090 using 8K-token prefill chunks. Once weights spill to SSD, baselines read most of the 155 GiB model from storage regardless of host RAM size, keeping TTFT persistently high. Across the full sweep from 128 to 32 GiB, baseline TTFT and TPOT increase by factors of approximately 1.1×–6.4× and 1.3×–7.0×, respectively. In contrast, our design coordinates cross-tier residency and execution, enabling TIDEWAY to cut SSD traffic and achieve lower TTFT and TPOT across all RAM budgets during both phases.

This performance degradation reflects limitations in both expert retention and execution. These systems fail to keep useful experts resident, and they execute inefficiently while paging. Figure 3 examines these limitations using a 32K prompt and 1K output tokens with 64 GiB of host RAM.

During prefill, each 8K-token chunk activates nearly the entire expert pool, requiring weights to stream through the GPU once per chunk. Existing systems were built on the assumption that host RAM is a backing cache of the VRAM. When the pool exceeds RAM, simply extending this hierarchy with LRU-style demand paging fails to preserve expert reuse across chunks. Under a cyclic scan, loading later-layer experts evicts earlier-layer experts, causing successive chunks to repeatedly reload the same weights from SSD. Figure 3(a) shows that SSD supplies 86–99% of the baselines' prefill expert traffic, although the 64 GiB RAM holds 44% of the pool. TIDEWAY keeps resident experts in place across the scan, and fetches only 52% from SSD.

Decode exposes a distinct execution bottleneck beyond weight I/O. CUDA Graphs typically reduce kernel launch overhead by replaying a captured, fixed-shape sequence, but that fixed shape complicates end-to-end capture with dynamic cross-tier paging. Existing systems thus fall back to eager execution for expert miss handling, increasing host dispatch and synchronization overhead. Figure 3(b) shows that CPU orchestration overhead and GPU stalls dominate decode time in most baselines, whereas llama.cpp incurs little launch overhead but spends 92% of decode time computing experts on the CPU. TIDEWAY minimizes CUDA Graph disruptions under expert paging, reduces expert I/O and CPU overhead, and achieves lower overall decode latency than all baselines.

In summary, accelerating MoE inference on consumer PCs requires making effective use of the memory capacity, while sustaining efficient execution across inference phases.

3 OVERVIEW

Motivated by the findings in §2, TIDEWAY is designed to enable frontier-scale MoE model inference on consumer PCs

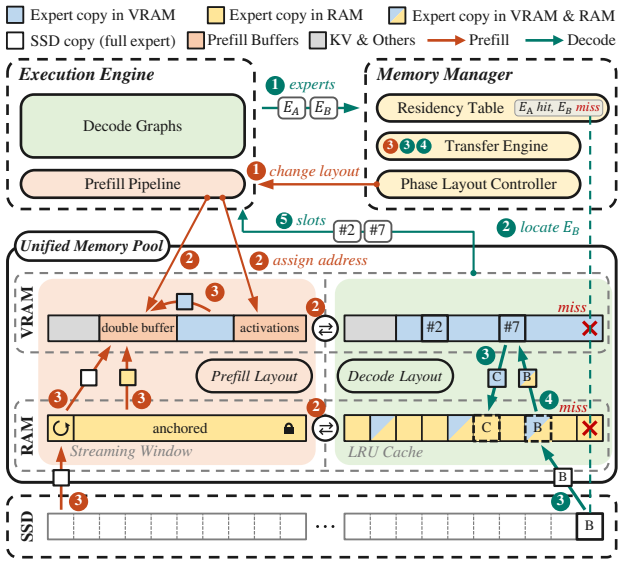

Figure 4. System architecture and phase-aware execution workflows of TIDEWAY across the three-tier memory hierarchy.

where model weights inevitably spill across VRAM, host RAM, and SSD. To overcome the structural bottlenecks of existing inference systems, TIDEWAY pursues two core design goals: (1) *maximizing effective memory capacity* across VRAM and host RAM while dynamically adapting memory layouts to prevent cross-tier thrashing, and (2) *sustaining efficient GPU execution* across both prefill and decode phases despite high-latency SSD paging.

Figure 4 presents the overall architecture of TIDEWAY. Across the underlying storage hierarchy, TIDEWAY abstracts GPU VRAM and host RAM as a *Unified Memory Pool*, while treating the SSD as an out-of-core backing store for spilled weights. Atop this hierarchy, TIDEWAY introduces two co-designed components: an *Execution Engine* and a *Memory Manager*. To regulate expert residency and cross-tier data movement, the memory manager tracks expert locations through the *Residency Table*, reconfigures pool partitions via *Phase Layout* controller, and dispatches asynchronous migrations through the *Transfer Engine*. Concurrently, the execution engine drives model computation through phase-specialized execution paths, pairing a streaming *Prefill Pipeline* with paging-aware *Decode Graphs*. Together, these two components coordinate memory layout adaptation and kernel execution across inference phases.

Prefill path. During prefill (red arrows), every chunk activates nearly all experts, so the memory pool is arranged for streaming rather than caching. The Phase Layout Controller ① first directs the Execution Engine to switch to the *Prefill Layout*. The switch ② reshapes both tiers: in VRAM, the Prefill Pipeline carves out an activation region and a double buffer for weight staging, borrowing the space from the decode expert cache; in RAM, the resident experts are anchored against eviction for reuse across chunks, and a small streaming window is reserved for experts arriving from SSD. The Transfer Engine ③ then fills the double buffer alternately with expert weights from VRAM, RAM, or SSD, overlapping weight delivery for layer $l{+}1$ with computation of layer l, with SSD-backed experts passing through the streaming window on their way up. When execution returns to decode, the anchors are released and the borrowed VRAM regions become available for expert caching again.

Decode path. When transitioning to decode (green arrows), TIDEWAY reconfigures the pool into the *Decode Layout*, reclaiming the borrowed VRAM regions as fixed-address expert slots and managing host RAM as an *LRU Cache*. Across this unified pool, experts may reside exclusively in VRAM, exclusively in RAM, or in both tiers. For each layer, the Decode Graphs ① resolve the routed experts (e.g., E_A and E_B) and query the *Residency Table*, which ② locates their weights across the memory hierarchy. In Figure 4, E_A hits directly in VRAM slot #2, whereas E_B misses in both VRAM and RAM. To accommodate E_B in VRAM slot #7, the Transfer Engine ③ concurrently demotes the VRAM-only victim C to RAM while fetching E_B from the SSD into RAM. Once E_B arrives in RAM, the engine ④ promotes it into VRAM slot #7, leaving E_B resident in both tiers. Finally, the Memory Manager ⑤ returns the resolved VRAM slot indices (#2 and #7) to the Execution Engine, allowing the *Decode Graphs* to replay captured kernels directly over those fixed slot addresses.

Together, these two paths preserve expert residency across phases. Prefill streams weights without flushing anchored experts in host RAM, while decode demotes evicted VRAM experts back to RAM rather than discarding them, minimizing subsequent SSD reloads. Realizing this architecture requires addressing two system challenges: managing cross-tier residency and layout transitions without wasting memory pool capacity (§4), and orchestrating asynchronous weight streaming and CUDA Graph replay while expert locations change at runtime (§5).

4 CROSS-TIER MEMORY MANAGER

The core objective of the Memory Manager is to maximize the effective capacity of VRAM and host RAM across inference phases to avoid costly SSD transfers. As shown in Figure 4, it achieves this through four co-designed mechanisms: a unified *Residency Table* (§4.1), *Phase Layout* management (§4.2), replica-aware replacement (§4.3), and an asynchronous *Transfer Engine* (§4.4).

4.1 Unified Memory Pool

Conventional offloading systems treat the memory tiers as a cache hierarchy, with VRAM caching RAM and RAM

caching the SSD, and let each tier replace its contents independently. On a consumer PC, however, the cost of crossing the tiers is sharply asymmetric. Bidirectional host-to-device (H2D) and device-to-host (D2H) transfers over PCIe complete several times faster than a direct read from the NVMe SSD. An expert resident in RAM remains a valuable fast copy that can be re-served to the GPU at low latency. Likewise, when the GPU evicts an expert that has no host copy, preserving that victim in RAM via a D2H writeback prevents a costly SSD reload on future accesses.

Consequently, rather than managing VRAM and RAM as independent caches, TIDEWAY unites them into a Unified Memory Pool. Within this pool, TIDEWAY reserves space for persistent model states and the KV cache, partitioning the remaining VRAM and host RAM into uniform expert slots. The pool's organization adapts to the available capacity. When host RAM alone fits all experts, it holds the complete working copy while VRAM caches active ones. When only the combined VRAM and RAM capacity can fit the model, TIDEWAY distributes experts across both tiers and deduplicates them to expand effective capacity. When the combined tiers fall short, experts are paged from the SSD on demand, shifting the manager's primary focus to preventing avoidable pool evictions. To coordinate residency across the pool, the Residency Table maintains a unified directory mapping experts to their VRAM and RAM slots, distinguishing cross-tier duplicates from an expert's last in-memory copy to guide layout adaptation and replacement.

4.2 Phase-aware Memory Layouts

Agent workflows alternate between prefill and decode, which favor different layout patterns in the same memory pool. Prefill scans a broad set of experts and requires space for weight streaming and activations, whereas decode accesses few experts per step and benefits from retaining its evolving working set near the GPU. TIDEWAY thus maintains phase-specific layouts that balance execution workspace against expert retention.

During prefill, the manager partitions available VRAM into expert storage, double-buffered weight staging, and activation workspace. In RAM, *anchors* protect reusable experts from being flushed and evicted by prefill scans (§6.2), while leaving a small *Streaming Window* to cycle SSD-backed weights. When decode resumes, the manager releases the anchors and reclaims the borrowed VRAM buffers back into fixed-address expert cache slots. Prefill thus uses VRAM for streaming throughput and decode reclaims it for cache capacity, while the anchored RAM-resident set survives across phases to serve subsequent decode requests (§4.3).

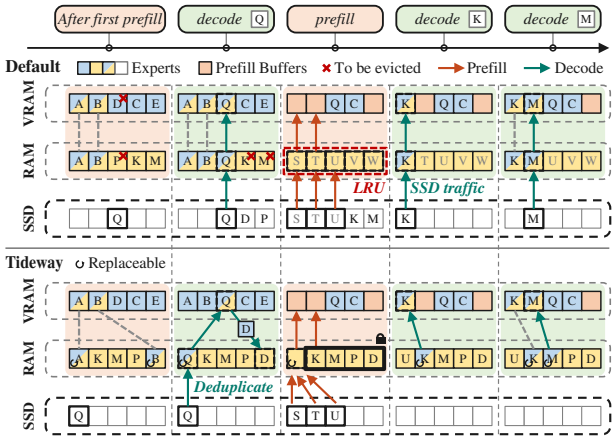

Figure 5. Comparison of cross-tier expert management between default per-tier LRU and TIDEWAY across phase transitions.

4.3 Replica-aware Replacement

Standard per-tier LRU replacement wastes capacity in a multi-tier pool for two reasons: promoting an expert from the SSD through RAM into VRAM leaves duplicated copies in both tiers, while independent eviction inside each tier is blind to cross-tier redundancy and may evict an expert's last copy in the pool. TIDEWAY therefore replaces independent per-tier eviction with a replica-aware policy that coordinates RAM reclamation and GPU victim preservation.

Lazy duplicate reclamation. When an expert is promoted from RAM to VRAM, TIDEWAY retains the RAM copy and marks it as *replaceable* (circular arrows in Figure 5). For RAM allocations, the manager uses free slots first, then reclaims eligible *replaceable* replicas, and finally falls back to LRU among other eligible residents while VRAM evicts by recency as usual. Once either copy is reclaimed, the surviving copy is no longer redundant. Meanwhile, to better balance unique pool capacity against repeated PCIe traffic, TIDEWAY exempts a small set of hot experts from duplicate reclamation, allowing them to remain dual-resident.

Selective victim writeback. VRAM is the executing tier and evicts far more often than RAM. Under deduplication, many of its victims are VRAM-only experts. Overwriting such a slot drops the expert's last in-pool copy and turns its next access into an SSD read. TIDEWAY looks one step further: before eviction, TIDEWAY checks whether a valid RAM copy remains. If so, it releases the GPU copy. If host RAM has a free or replaceable slot, TIDEWAY demotes the VRAM-only victim back to RAM over the almost idle D2H PCIe link. If neither is available, the victim simply leaves the pool without displacing another RAM-only expert.

Figure 5 illustrates how these policies cooperate with the phase-aware layouts. After the first prefill, both Default and TIDEWAY hold 8 distinct experts across 10 total slots,

with A and B duplicated in both tiers. When decode Q requests an SSD-only expert Q and evicts D from VRAM, Default's independent LRU drops D from VRAM and evicts P from RAM to hold Q, leaving Q duplicated and shrinking pool coverage to 7 distinct experts. In contrast, TIDEWAY reclaims the replaceable RAM copies of A and B while keeping their VRAM copies active, expanding coverage to 9 distinct experts with zero pool evictions. During the subsequent prefill, Default streams incoming weights (S–W) through its entire RAM under LRU, evicting the in-pool copies of K and M. TIDEWAY instead locks its unique RAM-only residents (K, M, P, D) with anchors and reuses Q's replaceable RAM slot as the streaming window to cycle S–U. When decode resumes for K and M, Default incurs costly SSD traffic to reload both weights, whereas TIDEWAY serves both hits directly from host RAM.

4.4 Asynchronous Expert Transfers

The Transfer Engine turns residency and layout decisions into cross-tier transfers. During prefill, it assembles each layer's weights in the double buffer. VRAM-resident experts are copied within the GPU, RAM-resident experts are transferred over PCIe, and SSD-only experts are loaded through the streaming window. Where possible, contiguous RAM slots are copied together in larger asynchronous transfers, reducing the overhead of issuing separate copies.

During decode, the engine transfers requested experts into their assigned VRAM slots and carries out selective victim writebacks. It uses asynchronous H2D copies to upload RAM-resident weights and D2H copies to preserve GPU-only victims. When multiple routed experts miss in the pool, the engine dispatches parallel SSD reads to overlap storage access latencies. For layouts that retain all experts at fixed locations in the pool, GPU kernels instead read selected weights directly from RAM into VRAM slots, avoiding per-expert CPU dispatch overhead.

As replacement and phase transitions reuse the same storage, TIDEWAY coordinates slot reuse with asynchronous transfer completion. The prefill buffers use two CUDA events. *Release* marks the end of computation using a buffer, and *ready* marks completion of its refill. The copy stream waits for release before loading new weights, while the compute stream waits for ready before using them. These dependencies prevent premature reuse while allowing the execution engine to overlap independent transfers and computation.

5 PAGING-AWARE EXECUTION ENGINE

While the Memory Manager determines where experts reside and how memory is partitioned, the *Execution Engine* schedules kernel execution and cross-tier transfers to hide I/O latency and minimize host dispatch overhead. It pairs a *Prefill Pipeline* that overlaps layer-wise weight streaming with computation (§5.1), and *Decode Graphs* that capture slot-addressed kernels (§5.2) to sustain graph replay under expert paging (§5.3).

5.1 Streaming Prefill Pipeline

The Prefill Pipeline processes each input chunk layer by layer over the double buffer provided by the Phase Layout Controller. Once the first layer's weights are staged, the engine submits its computation and asks the Transfer Engine to assemble the next layer in the other buffer. Delivery of layer $l{+}1$ thus runs concurrently with computation of layer l, and because a large prefill chunk makes each layer compute-intensive, the computation is long enough to cover the cross-tier transfers behind it.

The double buffer alternates as execution advances: the half consumed by layer l is refilled for layer $l{+}2$, and expert kernels always read their assigned half regardless of whether the weights came from VRAM, RAM, or SSD. To keep the slowest path off the critical path, the engine prepares layer $l{+}1$ after submitting the computation of layer l, including any required SSD reads and PCIe staging, and allows its delivery to overlap with the GPU work already in flight. When the last layer completes, the pipeline moves its persistent outputs out of the temporary workspace and returns the borrowed VRAM to the Memory Manager for decoding.

5.2 Piecewise CUDA Graph Capture

When expert paging requires host involvement, a decoder layer cannot be captured as a single monolithic CUDA Graph. Once the router computes active expert IDs on the GPU, the CPU needs to read back those IDs, query the Residency Table, assign target VRAM slots, and issue any required SSD reads and PCIe transfers. As these data-dependent host operations cannot reside inside a captured graph, default offloading engines abandon graph capture along the paged path and fall back to eager kernel dispatch, issuing a long sequence of individual kernel launches per layer. While TIDEWAY uses whole-step graphs when the entire expert pool fits in RAM, SSD-backed paging requires synchronizing routing decisions with the host. To retain graph execution without capturing separate graphs for different expert combinations, TIDEWAY decomposes each layer into reusable *piecewise CUDA Graphs*, isolating dynamic host paging and variable-width operations.

Under host paging, each layer is split into three segments. The first segment captures the attention prefix such as projections, normalization, and rotary embedding. Sparse attention stays eager. Capturing it would require padding the candidate set to a fixed width, and a different width can change the model's behavior. The second segment captures the front of the FFN, routing weights and expert IDs to-

gether with the shared expert, and ends where the host gets the expert routing. The third segment captures the back of the FFN, where routed experts are computed over slot IDs.

TIDEWAY captures these three segments for each layer once during the warmup phase, binding their inputs and outputs to persistent GPU buffers at fixed addresses. In particular, the third segment reads the normalized inputs, routing weights, and expert IDs directly from the buffers populated by the second segment, avoiding extra data copies across the paging boundary. As the attention segment updates the KV cache and internal states in place, warmup validation snapshots these states before running the reference check and restores them afterward so that capture leaves the model state intact. Once all layers are validated, the captured graphs are sealed, and subsequent tokens simply update the buffer contents in place and replay the segments in layer order without recapturing on expert misses.

5.3 Paging-Resilient Graph Replay

With the segments captured, the remaining question is how to execute when the routed experts may be anywhere in the hierarchy. TIDEWAY handles paging on the host between the segments, resolving routed experts into VRAM slots and fetching missing weights as needed. GPU transfers, slot-ID updates, and expert-graph replay are queued in order on the same CUDA stream. The host can therefore queue subsequent work before transfers finish, while the GPU executes expert kernels only after their weights and indices are ready. Every transfer in the flow is conditional on residency, so a layer whose experts all hit pays only the resolution and the upload, and a miss that reaches the SSD lengthens the handler without changing the execution path around it.

Figure 6 expands layer 2 to show this flow in action. The host ① reads back the expert IDs, and the Memory Manager ② resolves their residency and assigns VRAM slots. E_A hits in slot #2, whereas E_B is absent from the pool and is assigned slot #7, replacing GPU-only expert C. To prepare this slot, the Transfer Engine ③ overlaps C's asynchronous D2H writeback with the SSD read of E_B into RAM. Once the read completes, it ④ submits the H2D copy of E_B into slot #7. Finally, the paging handler ⑤ uploads the resolved slot IDs (#2 and #7) and queues the `experts` graph behind the transfers on the same stream. The host can then begin queuing layer 3 while the GPU completes layer 2.

6 EVALUATION

We implement TIDEWAY in Python and C, with GPU kernels written in Triton and CuTe DSL. The implementation comprises about 11.8K source lines, including 5.6K for memory management, 3.8K for execution and kernel adaptations, and 2.3K for backend integration and runtime support.

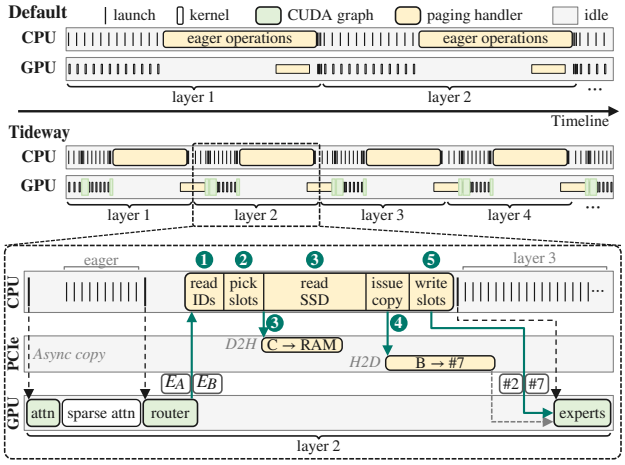

Figure 6. Decode execution timeline under default per-expert eager execution and TIDEWAY, and a zoomed-in view of layer 2 in TIDEWAY illustrating piecewise CUDA Graphs (attn, router, experts) coordinated with the five-step paging handler.

Experimental setup. All of our experiments run on a consumer desktop with a 32-core AMD CPU, 251 GiB RAM, a 32 GB NVIDIA GeForce RTX 5090 over PCIe 5.0, and an 8 TB WD_BLACK SN850X NVMe SSD, under Linux with NVIDIA driver 580.126, CUDA 13.0, PyTorch 2.11, and Triton 3.6. Measured pinned PCIe bandwidth reaches 57.6 GB/s H2D, 57.3 GB/s D2H, and 102.1 GB/s bidirectional, while SSD reads reach 6.8 GB/s. We evaluate **Gemma4-26B-A4B** (Gemma Team, 2026) in BF16, **Qwen3.6-35B-A3B** (Qwen Team, 2026) in BF16, and **DeepSeek-V4-Flash** (DeepSeek-AI, 2026) 0731 release in native FP4, capping host RAM at 16–128 GiB per model to vary memory pressure.

Baselines. We compare TIDEWAY against four baselines. **FreeToken** (Yang et al., 2026) v0.1.2 assumes full RAM residency, which we extend with demand SSD paging when weights exceed host RAM. **llama.cpp** (Gerganov et al., 2023) commit `67a17c1` runs with its native CPU–GPU execution and CUDA Graph support. As neither **SSD-LLaMA** (Liang et al., 2026) nor **FlashMoE** (Kim et al., 2026) is open-source, we reproduce their mechanisms on the unmodified FreeToken backend, implementing SSD-LLaMA's lossless expert compression with CUDA decompression in its all-GPU compute mode and FlashMoE's independently trained per-layer replacement predictors. All systems use the same models and RAM budgets and are tuned with author-recommended configurations.

Workloads and metrics. We evaluate *static shapes* with 4K–128K input tokens and 128–2K output tokens, covering 270 configurations across three models and their RAM budgets. We also replay eight multi-turn coding-agent sessions from *TraceLab* (Zhu et al., 2026), with growing contexts and repeated prefill–decode transitions. We report end-to-end

completion time, time-to-first-token (TTFT), and decode throughput (tokens/s), averaged within each model–RAM setting, with speedups relative to the fastest baseline. Unless otherwise stated, prefill uses 8K-token chunks.

6.1 End-to-End Performance

End-to-end completion time. Figure 7(a) compares average end-to-end request completion time across all three models and host-RAM budgets. Across all model-RAM configurations, TIDEWAY outperforms every baseline, achieving $1.13\times$–$3.18\times$ speedups ($1.97\times$ on average) on Static Shapes and $1.20\times$–$3.14\times$ on TraceLab. At the highest RAM budgets, where all experts fit in the unified memory pool, TIDEWAY still delivers $1.13\times$–$1.58\times$ speedups by eliminating per-expert CPU dispatch overhead and overlapping PCIe transfers. TIDEWAY's advantage becomes even more pronounced at the lower RAM budgets, where model weights unavoidably spill to the SSD. In these regimes, baselines suffer from severe SSD thrashing as independent per-tier caches retain duplicate copies and flush reusable experts during prefill. TIDEWAY minimizes costly SSD reloads, keeps CUDA Graph execution, and hides the remaining transfer latency, widening its speedup to $1.95\times$–$3.18\times$ on Static Shapes and $1.47\times$–$3.14\times$ on TraceLab.

Prefill TTFT. As shown in the top row of Figure 7(b), TIDEWAY reduces TTFT by $1.15\times$–$9.04\times$ across settings, with a $2.40\times$ average speedup over the fastest baseline. This improvement comes from two sources. The phase-aware layout (§4.2) keeps the RAM-resident experts anchored across chunks, reducing repeated SSD fetches (Figure 3(a)). The Streaming Prefill Pipeline further overlaps the remaining weight transfers with layer-wise GPU computation, using a specialized double buffer and kernels to reduce GPU stalls during weight transportation (§5.1).

Decode throughput. TIDEWAY also accelerates the decode phase. As shown in the bottom row of Figure 7(b), TIDEWAY achieves $1.20\times$–$2.28\times$ higher decode throughput ($1.61\times$ on average) than the fastest baseline. Phase-aware layouts and replica-aware replacement jointly reclaim prefill workspace and redundant-copy storage, expanding the effective expert capacity of VRAM and RAM and reducing expert I/O (§4.2–§4.3). When paging occurs, the execution engine retains CUDA Graph replay around the paging handler, reducing per-token kernel-launch and CPU orchestration overhead (§5.3), as shown in Figure 3(b).

6.2 Memory Residency under Phase Transitions

To understand why TIDEWAY sustains high efficiency across phases, we first inspect host-RAM slot accesses during a 32K-token prefill with 8K-token prefill chunks on DeepSeek-V4 with 64 GiB RAM in Figure 8. Under default per-tier LRU, streaming a full pass of layers exceeds host RAM capacity and overwrites existing slots in sequence, producing diagonal blue sweeps across the entire RAM pool. As each layer sweep evicts the experts needed by the next sweep, almost every slot access across all four chunks degrades into an SSD fetch. In contrast, TIDEWAY exhibits recurring RAM hits across chunks, with SSD fetches concentrated in a small streaming window. Anchors preserve reusable residents while the window absorbs the scan traffic (§4.2), allowing successive chunks to reuse cached weights.

Figure 9 further examines how prefill affects subsequent decoding by tracking per-step latency after each turn's prefill. Across all three RAM budgets, TIDEWAY starts at a lower initial latency and converges more quickly to a lower steady-state latency than every baseline. Even at 128 GiB RAM, where the model fits in memory, several baselines still suffer a sharp cold-start spike and remain elevated or exhibit step-to-step fluctuations. These observations are consistent with reduced disruption to expert residency during prefill, limiting the recovery cost when decode resumes.

6.3 Sensitivity Analysis on Prefill Chunk Size

Prefill chunk size decides how often the expert pool is swept per prompt, so we evaluate its effect on TTFT for DeepSeek-V4-Flash with a 32K-token input in Figure 10. Increasing the chunk size from 1K to 8K steadily reduces TTFT across all systems. Since each chunk accesses nearly all experts even at small chunk sizes, larger chunks require fewer model-wide weight sweeps, amortizing transfer costs over more tokens. In our experiments, we cap the prefill chunk size at 8K to limit activation-memory growth and preserve space for double buffering and resident experts on the GPU.

TIDEWAY achieves the lowest TTFT throughout, with its speedup over the fastest baseline increasing from $1.18\times$ at 1K to $1.57\times$ at 8K. Anchored RAM residency reduces repeated SSD reads across chunks (§4.2). As the chunk size increases toward 8K, each layer becomes progressively more compute-intensive, providing the Streaming Prefill Pipeline (§5.1) with a longer execution window per layer to hide lookahead SSD reads and double-buffered PCIe transfers behind GPU computation. In contrast, the baselines remain bottlenecked by unoverlapped I/O stalls and repeated cache thrashing across passes, causing TIDEWAY's relative advantage to widen at larger chunk sizes.

6.4 Ablation Study

To quantify the contribution of each proposed mechanism, Figure 11 presents a factor-by-factor breakdown on DeepSeek-V4-Flash with a 32K-token input, a 1K-token output, and 8K prefill chunks. We start from TIDEWAY-Base, which uses per-tier LRU eviction, synchronous expert loading, and eager execution. Enabling the Cross-tier Mem-

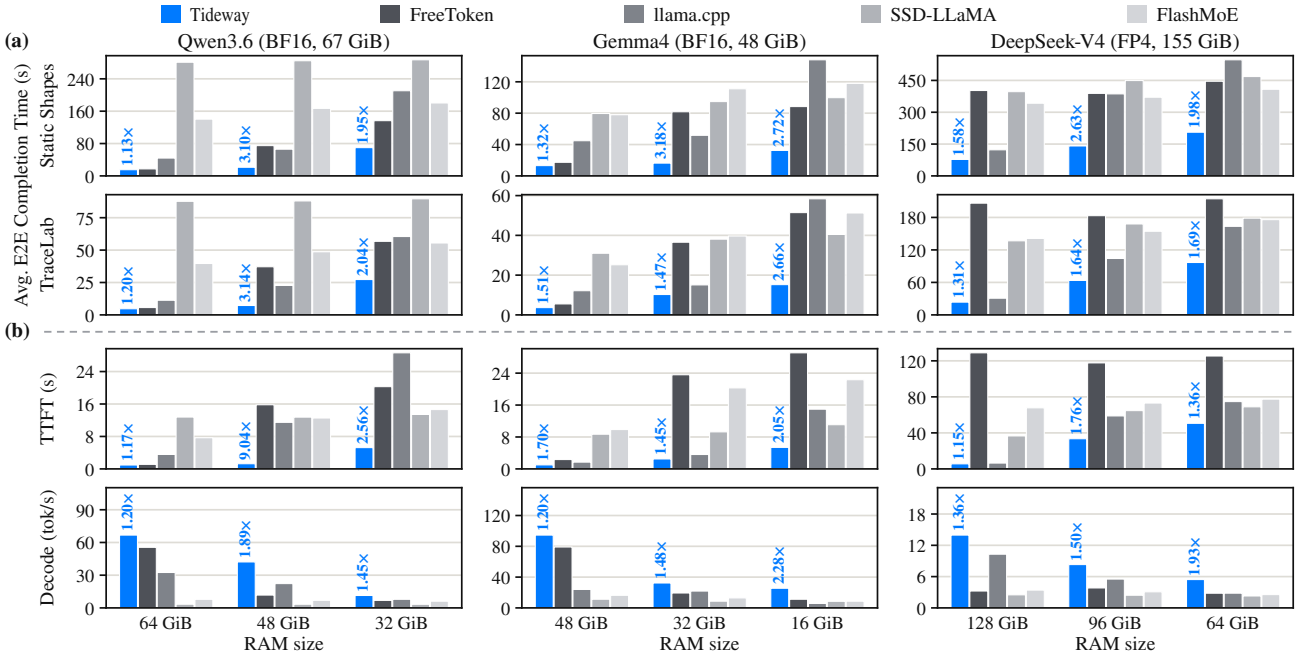

Figure 7. End-to-end inference performance on an RTX 5090 under three host-RAM budgets per model. (a) Average end-to-end completion time per request on Static Shapes and TraceLab. (b) Average time to first token (TTFT) and decode throughput on TraceLab. The number above each Tideway bar is its speedup over the fastest baseline.

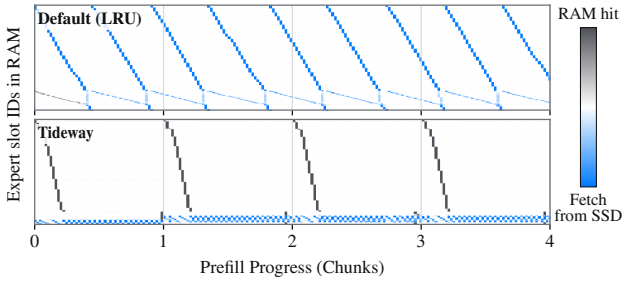

Figure 8. RAM slot residency while prefilling a 32K-token prompt in 8K-token chunks under default LRU and TIDEWAY on DeepSeek-V4-Flash at 64 GiB.

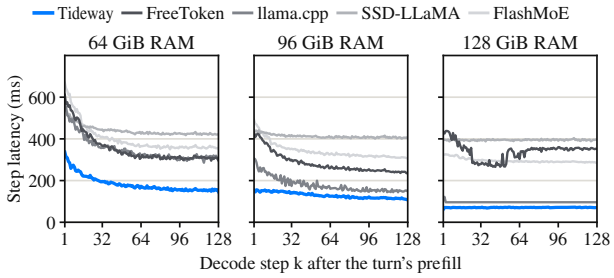

Figure 9. Per-step decode latency after a turn's prefill on DeepSeek-V4-Flash under three RAM budgets, averaged over TraceLab turns.

ory Manager in §4 protects reusable RAM residents during prefill and reclaims cross-tier duplicates, yielding a 1.23× end-to-end speedup. Adding asynchronous expert delivery and prefill pipelining from §4.4 and §5.1 overlaps SSD reads and double-buffered PCIe transfers with GPU computation, providing a further 1.36× gain. Finally, introducing piecewise decode graphs from §5.2–§5.3 removes host dispatch overhead under expert paging, delivering another 1.36× gain and bringing the overall end-to-end speedup to 2.28×.

The bottom panels of Figure 11 show the three mechanisms' speedup in different phases. For TTFT, the Memory Manager raises it to 1.55× by cutting SSD fetches, and asynchronous delivery to 2.94× by overlapping transfers and computation. Decode graphs bring no speedup to TTFT. For TPOT, the first two mechanisms lift it to 1.11× and 1.37×, and the bulk of the 2.04× comes from decode graphs reducing host dispatch stalls under expert paging.

7 DISCUSSION

Runtime overhead and SSD endurance. TIDEWAY's mechanisms cost little at run time. Capturing and validating the piecewise CUDA Graphs happens once during warmup and completes in a few seconds, and the persistent buffers and Residency Table take under 3% of the 32 GB of VRAM. All writeback operations, including selective D2H victim writebacks, are between VRAM and host RAM over PCIe, with the NVMe drive acting as a read-only backing store. As the unified pool and scan protection cut the bytes read from SSD per request (§2.2), TIDEWAY also reduces storage I/O pressure on consumer drives.

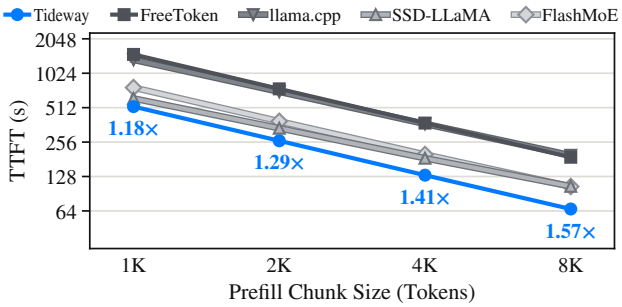

Figure 10. Prefill TTFT across chunk sizes (1K–8K tokens) for a 32K-token prompt on DeepSeek-V4-Flash with 64 GiB RAM.

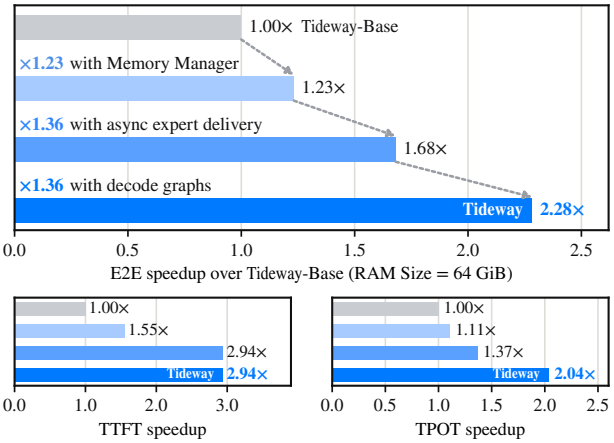

Figure 11. Ablation study of TIDEWAY on DeepSeek-V4-Flash with 32K-token input, 1,024-token output, and 8K prefill chunks.

KV cache as a competing tenant. TIDEWAY reserves KV capacity before allocating expert slots, so growing contexts leave less memory for expert residency. Extending the pool to manage KV blocks alongside experts could combine host offloading and prefix reuse (Zheng et al., 2024) to balance memory pressure against recomputation. Sparse KV access (Tang et al., 2024; Lee et al., 2024) could further reduce the transferred data. WiSP (Zhang et al., 2026) and MARS (Wang et al., 2026) show that the joint expert and KV working set bounds effective capacity on low-resource hardware, requiring admission control over this shared budget when serving concurrent requests.

Generality and orthogonal optimizations. Our design is orthogonal to weight quantization, speculative decoding, and lookahead expert prefetching. As shown in §6, TIDEWAY supports both BF16 and native FP4 quantized models, and can be combined with lossless storage compression when PCIe outpaces SSD bandwidth. Similarly, multi-token verification in speculative decoding maps naturally onto TIDEWAY's slot-addressed decode graphs by widening the fixed token dimension of the captured segments without recapturing on page misses.

8 Related Work

Offloading-based LLM inference. Early offloading engines such as FlexGen (Sheng et al., 2023) and DeepSpeed-Inference (Aminabadi et al., 2022) rely on static layer-wise schedules tailored to offline batch throughput. For sparse MoE models, Fiddler (Kamahori et al., 2025) and KTransformers (KVCache.AI Team, 2024) execute cold experts on the host CPU to save PCIe bandwidth, yet remain bottlenecked by host compute. Conversely, GPU-centric MoE systems including MoE-Infinity (Xue et al., 2024) and SwapMoE (Kong et al., 2025) overlap PCIe transfers via lookahead prefetching and activation-guided caching. While these designs assume sufficient host DRAM and fixed cache hierarchies, TIDEWAY manages VRAM and RAM as a unified pool to sustain execution under expert paging.

Expanding and sharing GPU memory. Recent systems also explore expanding VRAM capacity and supporting multiple applications on a single GPU. CUDA Unified Memory (NVIDIA, 2026) oversubscribes VRAM via page-granular demand paging. TGS (Wu et al., 2023), nvshare (Alexopoulos & Mitropoulos, 2024), and Nixie (Xu et al., 2026) enable multiple applications to share a single GPU, treating each workload's allocations as opaque memory blocks. TIDEWAY operates within a single model at expert granularity. Integrating TIDEWAY with these system-level multiplexers remains an interesting future direction.

Agentic workflows and graph execution. ReAct-style agents interleave generation with external feedback across turns (Yao et al., 2023), repeatedly invoking prefill and decode as contexts grow. Trending frontends such as Codex CLI (OpenAI, 2026) and OpenCode (OpenCode, 2026) further support spawning subagents, creating request dependencies at runtime. Parrot (Lin et al., 2024) exposes cross-request dataflow for application-level optimization, while Autellix (Luo et al., 2025) schedules calls using program-level progress. At the GPU level, FlashInfer (Ye et al., 2025) combines dynamic attention scheduling with CUDA Graph replay. Integrating workflow-aware scheduling with TIDEWAY could coordinate subagent concurrency and expert residency when multiple sessions share a local model.

9 Conclusion

Frontier MoE models are increasingly deployed to power local agents. We present TIDEWAY, a phase-aware tiered memory management and execution system for efficient agentic inference on consumer PCs. TIDEWAY unites GPU VRAM and host RAM into a unified memory pool, combining phase-specific layouts with replica-aware replacement and selective victim writeback to maximize distinct expert coverage and prevent prefill scan thrashing. Paired with

a streaming prefill pipeline and piecewise CUDA Graph replay under expert paging, TIDEWAY achieves 1.97× average end-to-end speedup on static workloads and up to 3.14× speedup on multi-turn agent traces over state-of-the-art baselines.

10 ACKNOWLEDGMENTS

This work was supported in part by the National Science Foundation (NSF) (Grant No. 2625580) and the Army Research Office (ARO) (Grant No. W911NF-23-2-0224).

REFERENCES

Alexopoulos, G. and Mitropoulos, D. nvshare: Practical GPU sharing without memory size constraints. In *Proceedings of the 46th IEEE/ACM International Conference on Software Engineering: Companion Proceedings (ICSE 2024)*, 2024.

Aminabadi, R. Y., Rajbhandari, S., Awan, A. A., Li, C., Li, D., Zheng, E., Ruwase, O., Smith, S., Zhang, M., Rasley, J., and He, Y. DeepSpeed-Inference: Enabling efficient inference of transformer models at unprecedented scale. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC 2022)*, pp. 1–15, 2022.

Anthropic. Claude 3.7 Sonnet and Claude Code, February 2025. URL https://www.anthropic.com/news/claude-3-7-sonnet. Accessed: 2026-10-05.

BOINC. BOINC project host statistics. https://boinc.berkeley.edu/, 2026. Host records with an RTX 5090 active in 2026, collected from the public host lists of PrimeGrid, MilkyWay@home, and Einstein@Home. Accessed: 2026-10-09.

DeepSeek-AI. DeepSeek-V4: Towards highly efficient million-token context intelligence. *arXiv preprint arXiv:2606.19348*, 2026.

Erdogan, L. E., Lee, N., Jha, S., Kim, S., Tabrizi, R., Moon, S., Hooper, C. R. C., Anumanchipalli, G., Keutzer, K., and Gholami, A. TinyAgent: Function calling at the edge. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing: System Demonstrations (EMNLP 2024)*, pp. 80–88, 2024.

Gemma Team. Gemma 4 technical report. *arXiv preprint arXiv:2607.02770*, 2026.

Gerganov, G. et al. llama.cpp: LLM inference in C/C++, 2023. URL https://github.com/ggml-org/llama.cpp. Accessed: 2026-10-05.

Kamahori, K., Tang, T., Gu, Y., Zhu, K., and Kasikci, B. Fiddler: CPU-GPU orchestration for fast inference of mixture-of-experts models. In *Proceedings of the 13th International Conference on Learning Representations (ICLR 2025)*, 2025.

Kim, B., Lee, J., Han, D., Yoo, H.-J., and Kim, S. FlashMoE: Reducing SSD I/O bottlenecks via ML-based cache replacement for mixture-of-experts inference on edge devices. *arXiv preprint arXiv:2601.17063*, 2026.

Kong, R., Li, Y., Wang, W., Kong, L., and Liu, Y. Serving MoE models on resource-constrained edge devices via dynamic expert swapping. *IEEE Transactions on Computers*, 74(8):2799–2811, 2025.

KVCache.AI Team. KTransformers: A flexible framework for experiencing cutting-edge LLM inference optimizations. https://github.com/kvcache-ai/ktransformers, 2024. Accessed: 2026-10-05.

Lee, W., Lee, J., Seo, J., and Sim, J. InfiniGen: Efficient generative inference of large language models with dynamic KV cache management. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 2024)*, 2024.

Liang, F., Shen, Y., Hu, J., Xu, J., Gao, H., Xu, M., and Meng, Z. SSD-LLaMA: SSD-native inference for trillion-parameter MoE at 1+ token/s on a consumer PC. *arXiv preprint arXiv:2609.18110*, 2026.

Lin, C., Han, Z., Zhang, C., Yang, Y., Yang, F., Chen, C., and Qiu, L. Parrot: Efficient serving of LLM-based applications with semantic variable. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 2024)*, 2024.

Luo, M., Shi, X., Cai, C., Zhang, T., Wong, J., Wang, Y., Wang, C., Huang, Y., Chen, Z., Gonzalez, J. E., and Stoica, I. Autellix: An efficient serving engine for LLM agents as general programs. *arXiv preprint arXiv:2502.13965*, 2025.

NVIDIA. CUDA C++ programming guide: Unified memory programming. https://docs.nvidia.com/cuda/cuda-c-programming-guide/, 2026. Accessed: 2026-10-24.

OpenAI. Introducing Codex, May 2025a. URL https://openai.com/index/introducing-codex/. Accessed: 2026-10-05.

OpenAI. Introducing Deep Research, February 2025b. URL https://openai.com/index/introducing-deep-research/. Accessed: 2026-10-05.

OpenAI. Codex documentation: Subagents. https://developers.openai.com/codex/subagents, 2026. Accessed: 2026-10-24.

OpenCode. OpenCode documentation: Agents. https://opencode.ai/docs/agents/, 2026. Accessed: 2026-10-24.

Qwen Team. Qwen3.6-35B-A3B. Hugging Face model card, https://huggingface.co/Qwen/Qwen3.6-35B-A3B, April 2026. Accessed: 2026-10-10.

Sheng, Y., Zheng, L., Yuan, B., Li, Z., Ryabinin, M., Chen, B., Liang, P., Ré, C., Stoica, I., and Zhang, C. FlexGen: High-throughput generative inference of large language models with a single GPU. In *Proceedings of the 40th International Conference on Machine Learning (ICML 2023)*, volume 202 of *Proceedings of Machine Learning Research*, pp. 31094–31116, 2023.

Song, Y., Mi, Z., Xie, H., and Chen, H. PowerInfer: Fast large language model serving with a consumer-grade GPU. In *Proceedings of the ACM SIGOPS 30th Symposium on Operating Systems Principles (SOSP 2024)*, pp. 590–606, 2024.

Tang, J., Zhao, Y., Zhu, K., Xiao, G., Kasikci, B., and Han, S. Quest: Query-aware sparsity for efficient long-context LLM inference. In *Proceedings of the 41st International Conference on Machine Learning (ICML 2024)*, 2024.

TrendForce. AI server demand sustains memory contract price increases in 4Q26, while consumer-side pressure persists, says TrendForce, September 2026. URL https://www.trendforce.com/presscenter/news/20260930-13258.html. Accessed: 2026-10-05.

UL Solutions. 3DMark benchmark results database. https://www.3dmark.com/search, 2026. Submitted results with an RTX 5090, Q3 2026. Accessed: 2026-10-09.

Valve Corporation. Steam hardware & software survey. https://store.steampowered.com/hwsurvey/, 2026. Q3 2026 results. Accessed: 2026-10-09.

Wang, Y., Ye, H., Xu, Y., Guo, C., Wei, C., Wang, Q., Li, D., Chen, T., Li, H. H., Zhuo, D., and Chen, Y. MARS: Efficient, adaptive co-scheduling for heterogeneous agentic systems. *arXiv preprint arXiv:2604.26963*, 2026.

Wu, B., Zhang, Z., Bai, Z., Liu, X., and Jin, X. Transparent GPU sharing in container clouds for deep learning workloads. In *20th USENIX Symposium on Networked Systems Design and Implementation (NSDI 2023)*, pp. 69–85, 2023.

Xie, T., Zhang, D., Chen, J., Li, X., Zhao, S., Cao, R., Hua, T. J., Cheng, Z., Shin, D., Lei, F., Liu, Y., Xu, Y., Zhou, S., Savarese, S., Xiong, C., Zhong, V., and Yu, T. OSWorld: Benchmarking multimodal agents for open-ended tasks in real computer environments. In *Advances in Neural Information Processing Systems (NeurIPS 2024)*, volume 37, 2024.

Xu, Y., Wang, Y., Ren, N., Chen, Y., and Zhuo, D. Nixie: Efficient, transparent temporal multiplexing for consumer GPUs. In *20th USENIX Symposium on Operating Systems Design and Implementation (OSDI 2026)*, pp. 2085–2101, 2026.

Xue, L., Fu, Y., Lu, Z., Mai, L., and Marina, M. MoE-Infinity: Offloading-efficient MoE model serving. *arXiv preprint arXiv:2401.14361*, 2024.

Yang, A., Li, A., Yang, B., Zhang, B., Hui, B., Zheng, B., Yu, B., Gao, C., Huang, C., Lv, C., Zheng, C., Liu, D., Zhou, F., Huang, F., Hu, F., Ge, H., Wei, H., Lin, H., Tang, J., Yang, J., Tu, J., Zhang, J., Yang, J., Yang, J., Zhou, J., Zhou, J., Lin, J., Dang, K., Bao, K., Yang, K., Yu, L., Deng, L., Li, M., Xue, M., Li, M., Zhang, P., Wang, P., Zhu, Q., Men, R., Gao, R., Liu, S., Luo, S., Li, T., Tang, T., Yin, W., Ren, X., Wang, X., Zhang, X., Ren, X., Fan, Y., Su, Y., Zhang, Y., Zhang, Y., Wan, Y., Liu, Y., Wang, Z., Cui, Z., Zhang, Z., Zhou, Z., and Qiu, Z. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025.

Yang, S., Fan, X., Pan, M., Xi, H., Wang, Z., Sun, S., Keutzer, K., Han, S., Zaharia, M., Xu, C., and Stoica, I. FreeToken: Efficient edge-native MoE serving with bandwidth-adaptive execution. *arXiv preprint arXiv:2608.16157*, 2026.

Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., and Cao, Y. ReAct: Synergizing reasoning and acting in language models. In *The Eleventh International Conference on Learning Representations (ICLR 2023)*, 2023.

Ye, Z., Chen, L., Lai, R., Lin, W., Zhang, Y., Wang, S., Chen, T., Kasikci, B., Grover, V., Krishnamurthy, A., and Ceze, L. FlashInfer: Efficient and customizable attention engine for LLM inference serving. In *Proceedings of the 8th Conference on Machine Learning and Systems (MLSys 2025)*, 2025.

Zhang, J., Wu, L., Darbari, M., and Hong, L. WiSP: A working-set view of mixture-of-experts serving on extremely low-resource hardware. *arXiv preprint arXiv:2606.21868*, 2026. doi: 10.48550/arXiv.2606.21868. URL https://arxiv.org/abs/2606.21868.

Zheng, L., Yin, L., Xie, Z., Sun, C., Huang, J., Yu, C. H., Cao, S., Kozyrakis, C., Stoica, I., Gonzalez, J. E., Barrett, C., and Sheng, Y. SGLang: Efficient execution of structured language model programs. In *Advances in Neural Information Processing Systems (NeurIPS 2024)*, volume 37, pp. 62557–62583, 2024.

Zhu, K., Jacob, M., Ma, C., Pan, Y., Wang, S., Krishnamurthy, A., and Kasikci, B. TraceLab: Characterizing coding agent workloads for LLM serving. *arXiv preprint arXiv:2606.30560*, 2026.